Programação Estruturada em C

Aula 5 — Funções, parâmetros e retorno

Prof. Me. Eng. Matheus Kotaki

A mudança desta aula

Até então...

- sequência de comandos
- decisões com if e switch
- repetições com while, do-while e for
- combinação dessas estruturas

A partir de hoje

- dividir o problema em responsabilidades
- dar nome às tarefas
- fazer as partes trocarem informações
- usar o main() como coordenador

Funções não substituem if, for ou while — elas ajudam a organizar o uso dessas estruturas.

Funcionar não significa estar bem organizado

```
int main() {
    float nota;

    do {
        printf("Digite a nota: ");
        scanf("%f", &nota);
    } while (nota < 0 || nota > 10);

    if (nota >= 7) {
        printf("Aprovado\n");
    } else if (nota >= 5) {
        printf("Recuperacao\n");
    } else {
        printf("Reprovado\n");
    }
    return 0;
}
```

A função main() reúne tarefas diferentes:

- ler e validar a nota
- decidir a situação
- apresentar o resultado

Identificando responsabilidades

Para processar uma nota:

Obter uma nota válida

Determinar a situação

Apresentar o resultado

Se cada tarefa tivesse um nome, quais nomes ajudariam a explicar o programa?

Função: uma tarefa nomeada do programa

```
void mostrarCabecalho() {
    printf("=== SISTEMA DE
NOTAS ===\n");
}
```

```
int main() {
    mostrarCabecalho();

    return 0;
}
```

Definir uma função ≠ executar a função

A função só executa quando ocorre uma chamada.

Fluxo de execução: main() → função → main()



A execução não ocorre “em paralelo”. Ela muda temporariamente de função e depois retorna ao ponto da chamada.

Anatomia de uma função

```
int maior(int a, int b) {
    if (a > b) {
        return a;
    }

    return b;
}
```

`int`	tipo do valor retornado
`maior`	nome da função
`int a, int b`	parâmetros
`return`	Envia um valor de volta

Ao criar uma função, faça três perguntas

1. **Qual é a responsabilidade?** → **nome da função**
2. **De quais informações precisa?** → **parâmetros**
3. **O que deve fornecer de volta?** → **retorno**

A partir de agora, evite concentrar toda a lógica dentro de main(). Funções passam a ser uma ferramenta permanente para organizar os programas.

A mesma função pode receber valores diferentes

```
int dobro(int valor) {
    return valor * 2;
}
```

dobro(3) valor recebe 3 → retorna 6

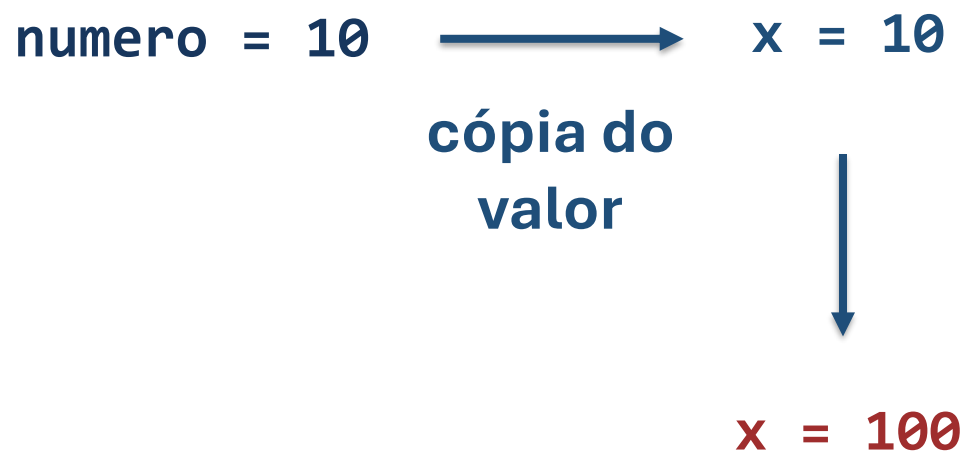
dobro(7) valor recebe 7 → retorna 14

dobro(10) valor recebe 10 → retorna 20

Passagem por valor: a função recebe uma cópia do valor

```
void alterar(int x) {
    x = 100;
}
```

```
int numero = 10;
alterar(numero);
printf("%d", numero);
```



Alterar x não altera automaticamente a variável numero.

Argumento e Parâmetro

```
int numero = 7;
int resultado = dobro(numero);
```

```
int dobro(int valor) {
    return valor * 2;
}
```

argumento

argumento: informação fornecida na chamada

parâmetro

parâmetro: variável usada pela função para receber essa informação

numero = 7 → valor = 7

return: a função retorna um valor para quem a chamou

```
float calcularMedia(float n1, float
n2) {
    return (n1 + n2) / 2;
}
```

```
float media;
media = calcularMedia(8.0, 6.0);
```

media = 7.0

retorna 7.0

Imprimir não é retornar

Apresentar um resultado

```
void mostrarDobro(int valor) {
    printf("%d", valor * 2);
}
```

printf() → envia informação para a tela

A função retorna nada

Produzir um resultado

```
int calcularDobro(int valor) {
    return valor * 2;
}
```

return → envia informação para quem chamou

Se o resultado precisa ser usado depois, retornar costuma ser mais útil do que apenas imprimi-lo.

Calcular não significa retornar

```
int dobro(int n);

int main() {
    int x = dobro(5);
    printf("%d", x);
    return 0;
}

int dobro(int n) {
    int resultado = n * 2;
}
```

O valor foi calculado...

...mas a função não informa esse valor a quem chamou.

```
int dobro(int n) {
    int resultado = n * 2;
    return resultado;
}
```

Quando usar void? Quando retornar?

A função executa uma ação?

Exemplos:

- mostrar uma mensagem
- exibir um relatório simples
- realizar uma sequência de saídas

→ void pode fazer sentido

A função descobre ou calcula algo?

Exemplos:

- calcular uma média
- determinar o maior valor
- verificar uma condição

→ retorno costuma ser útil

Protótipo: informar antes como a função será usada

```c
#include <stdio.h>

int maior(int a, int b);

int main() {
    int resultado = maior(10, 7);
    printf("%d\n", resultado);
    return 0;
}

int maior(int a, int b) {
    if (a > b) return a;
    return b;
}
```

1

Protótipo

informa nome, retorno e parâmetros

2

main()

usa a função

3

Definição

contém o código da função

Fluxo

```c
#include <stdio.h>

int dobro(int x);

int main() {
    int n = 5;

    int resultado = dobro(n);

    printf("%d\n", resultado);

    return 0;
}

int dobro(int x) {
    return x * 2;
}
```

1	**main**	`n = 5`	chama dobro(n)
2	**dobro**	`x = 5`	calcula x * 2
3	**dobro**	`x = 5`	retorna 10
4	**main**	`resultado = 10`	printf()

Escopo local: cada função possui suas próprias variáveis

```
void exemplo();

int main() {
    printf("%d", x);
    return 0;
}

void exemplo() {
    int x = 10;
}
```

O compilador encontra x aqui?

Não.

x foi declarada dentro de exemplo().
Ela é local àquela função.

Variáveis globais

```
int total;   // variável global

void atualizar() {
    total++;
}
```

C permite variáveis fora das funções.

- podem ser acessadas por várias funções
- facilitam o compartilhamento de estado
- mas também aumentam dependências ocultas

Nesta aula, priorize parâmetros e retorno para comunicar informações entre funções.

Funções podem conter condicionais

```
int ehPar(int numero) {
    if (numero % 2 == 0) {
        return 1;
    }

    return 0;
}
```

```
int n = 8;

if (ehPar(n)) {
    printf("Par\n");
}
```

Funções também podem conter repetição

```
int somarAte(int limite) {
    int soma = 0;

    for (int i = 1; i <= limite; i++)
{
        soma += i;
    }

    return soma;
}
```

Uma função pode tornar uma condição mais legível

```
int notaValida(float nota) {
    return nota >= 0 && nota <= 10;
}
```

Antes

```
while (nota < 0 || nota > 10) {
    // nova leitura
}
```

Com função

```
while (!notaValida(nota)) {
    // nova leitura
}
```

O objetivo não é “diminuir linhas”, mas expressar melhor o que está sendo verificado.

Exercício 1

Escreva um programa que leia um número inteiro. Crie uma função **chamada** dobro que **receba** esse número e **retorne** seu dobro.

Depois, no main(), mostre o resultado e informe se o dobro é positivo, negativo ou zero.

Exercício 2

Escreva um programa que leia o valor de uma compra.

Crie uma função **chamada** calcularDesconto que **receba** o valor da compra e **retorne** o valor do desconto de acordo com as regras:

- compras abaixo de R$ 100,00: sem desconto;
- compras de R$ 100,00 até R$ 499,99: 10% de desconto;
- compras de R$ 500,00 ou mais: 15% de desconto.

No main(), utilize o valor retornado pela função para calcular e mostrar:

- valor original da compra;
- valor do desconto;
- valor final a pagar.

Exercício 3

Escreva um programa que leia uma nota entre 0 e 10.

Crie uma função **chamada** notaValida que **receba** uma nota e **retorne**:

- 1, se a nota estiver entre 0 e 10;
- 0, caso contrário.

O programa deve continuar solicitando a nota enquanto o valor digitado for inválido.

Depois, crie outra função **chamada** mostrarSituacao que **receba** a nota válida e **mostre**:

- Aprovado, para nota maior ou igual a 7;
- Recuperacao, para nota maior ou igual a 5 e menor que 7;
- Reprovado, para nota menor que 5.

Exercício 4

Escreva um programa que leia 5 números inteiros.
Crie uma função **chamada** ehPar que **receba** um número e **retorne** 1 se ele for par e 0 caso contrário.

Ao final, o programa deve informar, **através de uma função que gere um relatório**:

- a soma de todos os números;
- quantos números são pares.

Usar um laço de repetição;
Usar acumulador e contadores;

Exercício 5

Escreva um programa que simule algumas operações de uma conta bancária.
A conta deve iniciar com saldo de **R$ 1.000,00**.

O programa deve exibir repetidamente o seguinte menu:
1 - Depositar
2 - Sacar
3 - Consultar saldo
0 – Sair

O programa deve continuar executando até que o usuário escolha a opção 0

Exercício 5 - Continuação

Regras – Use uma função para Depositar, Sacar e Consultar saldo:

- **Depositar:** solicitar um valor ao usuário. O depósito só deve ser realizado se o valor for maior que zero.
- **Sacar:** solicitar um valor ao usuário. O saque só deve ser realizado se o valor for maior que zero e menor ou igual ao saldo disponível.
- **Consultar saldo:** mostrar o saldo atual da conta.
- **Sair:** encerrar o programa.

Se o usuário digitar uma opção diferente das apresentadas no menu, o programa deve informar que a opção é inválida.

Se um depósito ou saque for inválido, o saldo deve permanecer inalterado.

Opcional: Trate caso o usuário digite um caractere na escolha do menu (crie uma função para limpar o buffer).

Exercício 5 - Continuação

Exemplo de funcionamento:

```
Saldo inicial: R$ 1000.00

1 - Depositar
2 - Sacar
3 - Consultar saldo
0 - Sair

Opcao: 1
Valor do deposito: 200
Deposito realizado.

Opcao: 2
Valor do saque: 150
Saque realizado.

Opcao: 3
Saldo atual: R$ 1050.00
```